DOCUMENT DE RENDU

README — Concert Agent

Présentation du projet, installation, configuration et scénarios de démonstration.

CA

Concert Agent · Octobre 2026

Application **Python / Flask** avec un **agent LLM** qui choisit lui-même ses tools, lit et écrit dans **SQLite**, et demande **confirmation avant toute écriture**.

Le projet reprend le thème « concerts » du site LEET et fusionne l'ancien site Spotify : l'utilisateur peut se connecter avec Spotify et importer ses artistes dans l'agent. Les données de concerts ne sont jamais inventées par le modèle : elles viennent de Ticketmaster ou de données de démonstration clairement signalées.

Fonctionnalités

- Agent avec tools réellement exécutés côté Python.
- 4 tools de lecture et 2 tools d'écriture.
- Confirmation obligatoire avant `ajouter_favori` et `ajouter_artiste_suivi`.
- Historique des interactions et traces de chaque tool dans SQLite.
- Interface web : chat, préférences, artistes suivis, favoris cliquables avec fiche détaillée, traces.
- Moteur de règles de secours si aucune clé LLM n'est configurée.
- Données de démonstration dynamiques, toujours étiquetées `source=demo`.
- Connexion Spotify (OAuth) et import des top artistes / écoutes récentes dans les artistes suivis.
- Jetons Spotify stockés côté serveur dans SQLite, jamais dans le cookie de session.

Démarrage rapide (Windows)

1. Double-cliquer sur `run.bat`.
2. Ouvrir **http://127.0.0.1:5000**.

Le script crée automatiquement `.venv`, installe `requirements.txt` et lance Flask.

Démarrage manuel

```
py -3 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe app.py
```

Configuration

Copier `.env.example` vers `.env`, puis renseigner les clés utiles.

LLM (Gemini)

```
GEMINI_API_KEY=ta_cle_google_ai_studio
LLM_PROVIDER=gemini
```

```
LLM_MODEL=gemini-3.5-flash-lite
```

Clé gratuite : https://aistudio.google.com/apikey

Sans clé, l'application fonctionne en **mode règles** : même interface, mêmes tools, mêmes traces, mais les décisions sont prises par des règles Python au lieu du LLM.

Ticketmaster

```
TICKETMASTER_API_KEY=ta_cle_ticketmaster
DEMO_FALLBACK=1
MODE_DEMO=0
```

Avec une clé, l'API Discovery est interrogée avec `locale=fr-fr` et `countryCode=FR`. L'application cherche d'abord l'attraction exacte pour éviter les homonymes, puis récupère ses événements en France. Sans clé, ou si aucun événement ne correspond, elle utilise les concerts de démonstration et l'indique dans la réponse. `MODE_DEMO=1` force ce mode.

Spotify

```
SPOTIFY_CLIENT_ID=ton_client_id
SPOTIFY_CLIENT_SECRET=ton_client_secret
REDIRECT_URI=http://127.0.0.1:5000/callback
```

1. Ouvrir le dashboard Spotify : https://developer.spotify.com/dashboard
2. Dans **Redirect URIs**, ajouter exactement `http://127.0.0.1:5000/callback`.
3. Dans **User Management**, ajouter les comptes autorisés (5 maximum en mode développement).
4. Redémarrer `run.bat`, puis cliquer sur **Se connecter avec Spotify**.

Spotify n'accepte plus l'alias `localhost` depuis fin 2025 : il faut l'adresse loopback explicite `127.0.0.1`. L'application redirige automatiquement `localhost` vers `127.0.0.1` pour éviter les erreurs de session.

L'import ajoute jusqu'à 30 artistes (top artistes + écoutes récentes), sans doublons. Le bouton **Importer mes artistes** permet de relancer l'import.

Les 3 scénarios de démonstration

1. **Recherche avec préférences**

   > Trouve un concert de mes artistes préférés en Auvergne-Rhône-Alpes le mois prochain.

L'agent appelle `consulter_preferences` puis `rechercher_concerts`, et affiche les concerts trouvés par Ticketmaster. Chaque concert est présenté avec sa date, son heure, son lieu, son prix et le lien de billetterie. Sans clé ou sans résultat, il affiche 3 concerts de démonstration en le signalant.

2. **Aucun résultat sans invention**

> Je veux voir un concert à Paris demain, max 5 €.

L'agent cherche réellement, ne trouve rien et répond qu'il ne peut pas inventer de concert.

3. **Écriture avec confirmation**

> Ajoute le concert de Gims à Décines à mes favoris.

L'agent prépare l'action, demande « Confirmer ? », puis exécute `ajouter_favori` uniquement après « Oui ».

Tests

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Les tests couvrent les 3 scénarios, la confirmation, le refus, l'historique, les préférences, les tools et l'intégration Spotify (sans appel réseau). Résultat attendu : `Ran 13 tests ... OK`.

Structure

```
concert-agent/
  app.py                 serveur Flask et API
  db.py                  SQLite, schéma, seed, historique
  agent/
    loop.py              boucle agent + confirmation
    llm.py               point d'appel LLM unique
    tools.py             tools et appels Ticketmaster
    spotify.py           OAuth et import des artistes Spotify
    regles.py            moteur de secours
    villes.py            villes, régions, normalisation
  templates/index.html   interface
  static/style.css       thème
  static/app.js          logique frontend
  tests/test_agent.py    tests des scénarios et de Spotify
  run.bat                lancement Windows
  render.yaml            déploiement Render
  Procfile               démarrage Gunicorn
```

Déploiement Render

Le fichier `render.yaml` décrit un service Python gratuit avec Gunicorn. Le dossier à publier est `concert-agent-code`, pas le projet Node `concert-alert`.

1. Mettre le code sur GitHub

1. Créer un compte sur https://github.com puis un dépôt nommé `concert-agent`.
2. Ouvrir le dossier `concert-agent-code`, sélectionner **tout son contenu** et le déposer à la racine du dépôt GitHub via **Add file > Upload files**.
3. Ne jamais envoyer `.env`, `data/` ni `.venv/` : ils contiennent les secrets et les données locales.
4. Valider avec **Commit changes**.

Le dépôt doit contenir `render.yaml`, `requirements.txt`, `app.py`, `agent/`, `static/` et `templates/` à sa racine.

2. Créer le service sur Render

1. Créer un compte sur https://render.com puis choisir **New > Blueprint**.
2. Connecter GitHub et sélectionner le dépôt `concert-agent`.
3. Render détecte `render.yaml` et propose un service gratuit. Valider avec **Apply**.
4. Renseigner les variables demandées avec les valeurs de ton `.env` local : - `GEMINI_API_KEY` - `TICKETMASTER_API_KEY` - `SPOTIFY_CLIENT_ID` - `SPOTIFY_CLIENT_SECRET` - `REDIRECT_URI` : laisser vide pour le premier déploiement.
5. Ne pas modifier `FLASK_SECRET` : Render le génère automatiquement.
6. Attendre la fin du déploiement et copier l'URL publique affichée, du type `https://concert-agent-xxxx.onrender.com`.

3. Configurer Spotify et tester

1. Dans Render > service > **Environment**, mettre `REDIRECT_URI` à `https://<ton-url>.onrender.com/callback`, puis enregistrer et attendre le redéploiement.
2. Dans https://developer.spotify.com/dashboard > application > **Redirect URIs**, ajouter exactement la même URL, caractère pour caractère.
3. Ouvrir l'URL Render : la recherche, les favoris et les liens de billetterie fonctionnent sans Spotify.
4. Pour que ton ami utilise Spotify, ajouter son compte dans **User Management** du dashboard Spotify (mode développement limité à 5 comptes).

Limites du plan gratuit

- Le service s'endort après inactivité : le premier chargement peut prendre environ une minute.
- Le disque est éphémère : la base SQLite est recréée à chaque redéploiement ou redémarrage. Les favoris et l'historique ne sont pas permanents.

- Pour conserver les données, ajouter un disque persistant payant et définir `CONCERT_DB_PATH` vers ce disque, ou migrer vers Postgres.
- La base locale n'est pas publiée : le service démarre avec les données de démonstration. Un import Spotify est nécessaire pour retrouver tes artistes.

Limites connues

- La couverture Ticketmaster en France est partielle : certains artistes n'ont aucun événement dans l'API. Dans ce cas, le fallback de démonstration est utilisé et signalé. La recherche réelle nécessite `locale=fr-fr` ; sans ce paramètre, l'API ne renvoie que quelques événements sportifs français.
- Le LLM réel a été validé localement avec `gemini-3.5-flash-lite` ; sans clé, le mode règles reste le fallback. Ce modèle est préféré à `gemini-3.6-flash`, dont le quota gratuit de 20 requêtes/jour est trop vite épuisé.
- Les concerts `source=demo` ne sont **pas** de vrais événements.
- Spotify en mode développement limite l'application à 5 comptes ; il faut ajouter chaque compte dans le dashboard.
- Le parcours OAuth réel a été validé localement après ajout de l'URI `http://127.0.0.1:5000/callback` dans Spotify ; les tests automatiques ne font aucun appel réseau.
- Pour une mise en production, utiliser `render.yaml` ou un vrai serveur WSGI, un `FLASK_SECRET` robuste et un disque persistant pour SQLite.